

Module 4 — Payments & Memberships — Viva File Guide (Visal)

Everything you own, ranked by how likely you are to be asked to open it and explain it. Backend is ~1,100 lines total, frontend ~1,600. It is genuinely small enough to know cold.

Tier 1 — Know these line by line (5 files, ~580 lines)

If the panel says "show me where X happens", it is almost certainly in one of these.

1. `server/src/modules/payments/payment.service.js` (209 lines)

The single most important file you own. Contains:

- `assertPayable()` — the guard: booking exists, belongs to this client, is `ACCEPTED`, not already paid.
- `payWithCash()` — one insert, no fee, no revenue, no earning record.
- `payWithCard()` — charge first, then a **transaction** writing payment + commission revenue + provider earning together.
- `getProviderTotalEarningsSummary()` — the figure Maheli's dashboard displays.
- `getCardPaymentAmount()` — the locked amount Dinuka's invoice module reads.

2. `server/src/modules/memberships/membership.service.js` (187 lines)

- `getMembershipOverview()` — everything the Subscription page needs in one call.
- `purchaseMembership()` — blocks renewal while ACTIVE, charges, then a transaction: supersede old → insert membership → insert payment → insert revenue → restore online.
- `runMembershipExpiryCheck()` — expire → email → force offline after grace.

3. `server/src/modules/memberships/membershipPricing.service.js` (37 lines)

Small but a favourite viva target. The provider **never sends a price**; the backend derives it from category count + whether this is the first payment, reading `membership_pricing_rules`.

4. `database/schema.sql` — the `booking_payments` table (around line 564)

```
platform_fee numeric(12,2) GENERATED ALWAYS AS (  
  CASE WHEN payment_method = 'CARD' THEN round(service_amount * 0.05, 2)  
  ELSE 0 END) STORED
```

The 5% is a generated column — the database computes it, not JavaScript, not the browser. `total_amount` is generated the same way. Also note the `*_snapshot` columns (payee bank, client name, provider name, category): the payment record keeps its own copy so a later profile edit can never rewrite payment history.

5. `server/src/modules/payments/payment.queries.js` (89 lines)

All the SQL behind the above. Parameterised throughout.

Tier 2 — Know what they do and be able to find things in them

File	Why it matters
<code>payments/payment.routes.js</code>	5 endpoints + <code>requireClient</code> / <code>requireProvider</code> guards
<code>memberships/membership.routes.js</code>	<code>GET /me</code> , <code>POST /purchase</code>
<code>payments/paymentGateway.service.js</code>	Simulated gateway. Cards starting <code>4000</code> are declined on purpose for the demo. Returns only a reference, brand and last four
<code>reviews/review.service.js</code>	Submitting the review is what flips the booking to <code>COMPLETED</code> and recalculates the average rating
<code>payment-settings/paymentSettings.service.js</code>	Bank details that autofill the payee section
<code>jobs/membershipExpiry.job.js</code>	The scheduled job that calls <code>runMembershipExpiryCheck()</code>
<code>database/migrations/007_membership_tables.sql</code>	Membership tables
<code>database/migrations/009_payment_and_review_tables.sql</code>	Payment + review tables
<code>database/migrations/019_notification_invoice_earning_tables.sql</code>	<code>provider_earnings</code>

Your tables: `booking_payments` , `provider_earnings` , `revenue_entries` , `membership_payments` , `provider_memberships` , `membership_pricing_rules` , `provider_payment_settings` , `reviews` .

Tier 3 — Frontend (know the flow, not every line)

File	Role
<code>client/pages/BookingPaymentPage.jsx</code> (202)	The payment page — method selection, orchestration
<code>client/components/CardPaymentForm.jsx</code> (175)	Card fields + confirmation popup
<code>client/paymentMath.js</code>	Fee preview for display only — the backend recomputes it
<code>client/pages/SubmitReviewPage.jsx</code> (146)	Review after payment
<code>provider/pages/SubscriptionPage.jsx</code> (135)	Membership status, pricing, history
<code>provider/components/MembershipPaymentForm.jsx</code> (124)	Membership card payment
<code>provider/components/PaymentSettingsSection.jsx</code> (158)	Bank details form

Questions you should have a crisp answer to

"Where is the 5% calculated?" In the database, as a `GENERATED ALWAYS ... STORED` column on `booking_payments` . The frontend shows a preview for the user's benefit, but the stored figure is computed by Postgres from `service_amount` and `payment_method` . A tampered frontend value cannot change what is recorded.

"What stops a payment being recorded without revenue, or vice versa?" `payWithCard()` opens a transaction and writes the payment, the commission revenue row and the provider earning row inside it, with `ROLLBACK` on any error. They cannot drift apart.

"Why charge the card before the transaction?" So a declined card records nothing at all. The `AppError(..., 402)` is thrown before any insert.

"Why is Cash different?" HomeHero never touches the money, so there is no commission, no earning record, and `service_amount` is stored as `NULL`. The only record of a cash amount anywhere in the system is the figure the provider types into their invoice — and that is explicitly not HomeHero-verified.

"Where is the card number stored?" Nowhere. The gateway returns only a reference, brand and last four. The full number never leaves the request. (See `paymentGateway.service.js`.)

"What happens if the notification fails after a successful payment?" Nothing — it is wrapped in its own `try/catch` after the `COMMIT`, deliberately, so a notification failure can never undo a payment. Same principle the spec applies to emails.

"How does a provider get forced offline?" `runMembershipExpiryCheck()`, called by `membershipExpiry.job.js`: expire memberships past their date and email those providers, then force offline anyone whose 3-day grace has also run out. Renewing calls `restoreOnlineIfForcedOffline()` in the same transaction as the purchase.

"Can a provider renew early and stack two memberships?" No. `purchaseMembership()` rejects with 409 while the current membership is `ACTIVE`.

"How is the price decided?" `getMembershipQuote()` reads the category count and the number of past memberships, then looks up `membership_pricing_rules`. LKR 4,999 for one category; for two, LKR 9,998 first payment and LKR 7,498.50 per renewal. The client never sends an amount.

Database objects you own

Verified against `database/schema.sql` and the live DB (`HomeHero_latest V10.backup`).

Tables (8)

Table	PK	Purpose
<code>booking_payments</code>	<code>booking_payment_id</code>	One row per paid booking, Cash or Card
<code>provider_earnings</code>	<code>provider_earning_id</code>	What the provider earns per Card payment
<code>revenue_entries</code>	<code>revenue_entry_id</code>	HomeHero's own income (commission + membership)
<code>provider_memberships</code>	<code>membership_id</code>	One row per membership period
<code>membership_payments</code>	<code>membership_payment_id</code>	The card payment for a membership
<code>membership_pricing_rules</code>	<code>membership_pricing_rule_id</code>	Reference data: 1-category and 2-category pricing
<code>provider_payment_settings</code>	<code>provider_user_id</code>	Provider bank details (payee autofill)
<code>reviews</code>	<code>review_id</code>	Client review; submitting it completes the booking

Generated columns — `booking_payments`

```
platform_fee  GENERATED ALWAYS AS (CASE WHEN payment_method='CARD'
                                THEN round(service_amount*0.05,2) ELSE 0 END) STORED
total_amount  GENERATED ALWAYS AS (service_amount + platform_fee) STORED
```

The 5% and the total are computed and stored by Postgres. Application code writes neither — `insertBookingPayment()` reads them back with `RETURNING`.

CHECK constraints

Constraint	Table	Rule
<code>chk_one_revenue_source</code>	<code>revenue_entries</code>	Exactly one of <code>booking_payment_id</code> / <code>membership_payment_id</code> is set
<code>revenue_entries_amount_check</code>	<code>revenue_entries</code>	<code>amount > 0</code>
<code>provider_earnings_amount_check</code>	<code>provider_earnings</code>	<code>amount > 0</code>
<code>chk_grace_after_expiry</code>	<code>provider_memberships</code>	<code>grace_ends_at >= expires_at</code>
<code>..._category_count_snapshot_check</code>	<code>provider_memberships</code>	Category count is 1 or 2
<code>..._category_count_check</code>	<code>membership_pricing_rules</code>	Category count is 1 or 2
<code>reviews_rating_check</code>	<code>reviews</code>	Rating between 1 and 5

UNIQUE constraints — these are your real business rules

Constraint	Enforces
<code>booking_payments_booking_id_key</code>	One payment per booking — double payment is impossible
<code>booking_payments_gateway_reference_key</code>	No duplicate gateway reference
<code>provider_earnings_booking_payment_id_key</code>	One earning record per payment
<code>membership_payments_membership_id_key</code>	One payment per membership
<code>membership_payments_gateway_reference_key</code>	No duplicate gateway reference
<code>provider_memberships_provider_user_id_membership_sequence_n_key</code>	Sequence numbers don't repeat per provider
<code>membership_pricing_rules_category_count_key</code>	One pricing rule per category count
<code>reviews_booking_id_key</code>	One review per booking

Indexes (13)

Partial unique indexes — the clever ones, worth calling out:

Index	What it guarantees
<code>uix_one_active_membership</code>	<code>UNIQUE (provider_user_id) WHERE status='ACTIVE'</code> — a provider cannot hold two active memberships, enforced by the database, not just by the 409 check in <code>purchaseMembership()</code>
<code>uix_rev_booking_payment</code>	One commission revenue row per booking payment
<code>uix_rev_membership_payment</code>	One membership revenue row per membership payment
<code>uix_reviews_booking_id</code>	One review per booking

Plain B-tree indexes:

Index	Table (column)	Serves
<code>ix_bp_booking_id</code>	<code>booking_payments (booking_id)</code>	(redundant — see gaps)
<code>ix_bp_paid_at</code>	<code>booking_payments (paid_at)</code>	Date-range reporting
<code>ix_mpy_paid_at</code>	<code>membership_payments (paid_at)</code>	Date-range reporting
<code>ix_pe_provider_user_id</code>	<code>provider_earnings (provider_user_id)</code>	Total Earnings lookup
<code>ix_pm_provider_user_id</code>	<code>provider_memberships (provider_user_id)</code>	Membership lookup
<code>ix_pm_expires_at</code>	<code>provider_memberships (expires_at)</code>	Expiry job
<code>ix_pm_grace_ends_at</code>	<code>provider_memberships (grace_ends_at)</code>	Forced-offline job
<code>ix_pm_membership_status</code>	<code>provider_memberships (membership_status)</code>	Status filtering

Foreign keys (10)

`booking_payments` → `bookings` · `provider_earnings` → `booking_payments`, `service_provider_profiles` ·
`revenue_entries` → `booking_payments`, `membership_payments` · `provider_memberships` → `service_provider_profiles`,
`membership_pricing_rules` · `membership_payments` → `provider_memberships` · `provider_payment_settings` →
`service_provider_profiles` · `reviews` → `bookings`

Views you use

View	Used by
<code>vw_current_provider_membership</code>	Yours. <code>DISTINCT ON (provider_user_id)</code> picks the latest membership and computes <code>is_in_grace_period</code> . Used by your <code>membership.queries.js</code> and also by Maheli and Vihas
<code>vw_monthly_revenue</code>	Reads your <code>revenue_entries</code> , splitting membership vs commission income per month. Feeds Vihas's 6-month chart and earnings report

Triggers, functions, stored procedures — you own none

The live database has exactly 6 trigger functions and 6 triggers, and **not one is on your tables**: they belong to bookings, client/provider profiles, portfolio images and provider categories.

Be ready to say why, because it sounds like a gap and isn't:

My module's invariants are enforced declaratively rather than procedurally — generated columns for the fee, partial unique indexes for "one active membership" and "one revenue row per payment", CHECK constraints for amounts and dates, and multi-statement writes wrapped in explicit transactions in the service layer. A trigger would have hidden that logic from the code that owns it.

Trap to avoid: `database/functions/purchase_membership.sql` and `database/triggers/updated_at_triggers.sql` exist but are **0-byte placeholder files** — the whole `database/functions/` and `database/triggers/` folders are empty scaffolding from early planning. If a panelist opens `purchase_membership.sql` expecting a stored procedure, say plainly that the folders were scaffolding and the logic lives in `membership.service.js` inside a transaction.

Gaps I found

- `ix_bp_booking_id` is redundant.** `booking_payments.booking_id` already has a UNIQUE constraint, and Postgres creates an index for that automatically. This second index costs write time and storage for no gain. Safe to drop.
- `updated_at` has no trigger** on `provider_memberships` or `provider_payment_settings`. Not currently a bug — every `UPDATE` in your queries sets `updated_at = now()` by hand — but it depends on nobody forgetting. A **BEFORE**

`UPDATE` trigger would make it structural. (This is what the empty `updated_at_triggers.sql` was meant to hold.)

3. Indexing is otherwise complete — every column your queries filter or join on is covered.

One weak spot to be ready for

There are no automated tests for this module — `server/tests/` contains only `setup.js`. If asked how you verified the payment logic, answer honestly: manual testing through the UI, including the deliberate `4000...` decline card for the failure path. If you want to strengthen this before the viva, the highest-value tests would be `getMembershipQuote()` (pure pricing logic, easy to test) and `assertPayable()` (the payment guard).